RIHLA MR

Plateforme de transport interurbain

Dossier de conception fonctionnelle et technique

Une plateforme B2B2C qui connecte voyageurs, compagnies, agences et agents d’embarquement autour d’un stock unique de sièges, de la réservation au billet QR.

Vision produit

“Un seul système de disponibilité, réservation, paiement et embarquement, que le billet soit vendu en ligne ou au guichet.”

Version 1.0 — Octobre 2026

Document de cadrage produit, domaine, architecture, API, sécurité, tests, CI/CD et roadmap

Sommaire

1	Vision, positionnement et utilisateurs
2	MVP et parcours utilisateur
3	Règles métier et cycles de vie
4	Modèle de domaine et données PostgreSQL
5	Architecture backend et frontend
6	API REST et contrats
7	Paiements, QR et notifications
8	Kafka et architecture événementielle
9	Sécurité, multi-tenancy et conformité
10	Tests, concurrence et qualité
11	Observabilité, performance et déploiement
12	Roadmap Phase 0 → Production
13	Critères de succès du projet

1. Vision, positionnement et utilisateurs

Rihla MR est une plateforme de mobilité interurbaine conçue pour servir à la fois les voyageurs et les opérateurs de transport. Le produit ne se limite pas à la réservation en ligne : il fournit aussi aux compagnies un système d'exploitation quotidien pour leurs agences, véhicules, départs, sièges, ventes et embarquements.

> **Principe directeur**
>
> Le stock de sièges est unique. Une place vendue sur le web, au téléphone ou au guichet doit être immédiatement indisponible partout ailleurs.

1.1 Problèmes résolus

- Pour le voyageur : difficulté à comparer compagnies, horaires, prix, disponibilité et points de départ.
- Pour la compagnie : ventes dispersées entre guichets, agences, téléphone et web, avec risque de double réservation.
- Pour l'agent : absence d'un manifeste centralisé, difficulté de contrôle des billets et de suivi des encaissements.
- Pour la plateforme : besoin d'un modèle multi-tenant garantissant que chaque compagnie ne voit que ses propres données.

1.2 Positionnement produit

```
PASSAGERS
  ↓ recherche / comparaison / réservation / paiement / billet
RIHLA MR
  ↓ planning / véhicules / sièges / agences / ventes / embarquement
COMPAGNIES DE TRANSPORT
```

1.3 Rôles

Rôle	Responsabilités principales
PASSENGER	Recherche, réservation, paiement, consultation et annulation.
COMPANY_OWNER	Administration complète de la compagnie.
COMPANY_MANAGER	Routes, trajets, véhicules, tarifs, personnel et rapports.
AGENCY_AGENT	Vente guichet, encaissement cash, gestion des réservations.
BOARDING_AGENT	Manifeste, scan QR, check-in et embarquement.
PLATFORM_ADMIN	Validation compagnies, supervision, sécurité et audit.
SUPPORT_AGENT	Support limité aux voyageurs et opérateurs.

2. MVP et parcours utilisateur

2.1 MVP voyageur

- Interface français / arabe avec RTL.
- Recherche origine → destination, date et nombre de passagers.
- Comparaison des départs par heure, prix, compagnie et disponibilité.
- Sélection du ou des sièges.
- Réservation sans compte avec téléphone de contact.
- Paiement cash ou fournisseur simulé au MVP.
- Billet QR et consultation par référence.
- Annulation selon une politique configurable.

2.2 MVP opérateur

- Authentification sécurisée des employés.
- Gestion des agences, véhicules et plans de sièges.
- Gestion des routes, trajets et tarifs.
- Ventes guichet partageant le même inventaire que le web.
- Manifeste passagers par trajet.
- Scan QR et embarquement.
- Dashboard simple : départs, ventes, taux de remplissage, CA.

2.3 Parcours voyageur

```
Accueil
  ↓
Recherche NKC → NDB
  ↓
Résultats / filtres
  ↓
Détail trajet
  ↓
Choix siège
  ↓
Informations passager
  ↓
Paiement / cash
  ↓
Confirmation + QR
  ↓
Check-in / embarquement
```

2.4 Écrans clés

Écran	Contenu essentiel
Accueil	Origine, destination, date, passagers, langue.
Résultats	Compagnie, horaire, prix, durée, places restantes.
Détail trajet	Agence, véhicule, équipements, politique, prix.

Choix des sièges	Plan visuel : disponible / sélectionné / occupé / bloqué.
Checkout	Contact, passagers, récapitulatif, moyen de paiement.
Billet	Référence, trajet, siège, QR signé.
Operator Dashboard	Départs, ventes, occupation, CA.
Boarding	Scan QR, statut ticket, confirmation embarquement.

3. Règles métier et cycles de vie

3.1 Inventaire de sièges

Chaque trajet possède son propre inventaire de sièges. La disponibilité n'est jamais calculée uniquement côté frontend : PostgreSQL reste la source de vérité transactionnelle.

```
AVAILABLE → HELD → BOOKED → CHECKED_IN → BOARDED
     ↘ expiration → AVAILABLE
BLOCKED = siège indisponible administrativement
```

> **Règle de concurrence**
>
> Si 100 clients tentent de réserver le dernier siège au même instant, une seule transaction doit réussir. Les autres reçoivent SEAT_UNAVAILABLE.

3.2 Cycle Booking

```
PENDING → HELD → AWAITING_PAYMENT → CONFIRMED → CHECKED_IN → BOARDED → COMPLETED
                  ↘ PAYMENT_FAILED
HELD → EXPIRED
CONFIRMED → CANCELLED
```

3.3 Cycle Trip

```
DRAFT → SCHEDULED → BOARDING → DEPARTED → ARRIVED → COMPLETED
              ↘ DELAYED
              ↘ CANCELLED
```

3.4 Cycle Payment

```
CREATED → PENDING → SUCCEEDED
                 ↘ FAILED
SUCCEEDED → REFUND_PENDING → REFUNDED
```

3.5 Règles critiques

- Un siège ne peut appartenir qu'à une réservation active à la fois.
- Un hold de siège expire automatiquement après une durée configurable (ex. 10 min).

- Une réservation ne devient CONFIRMED qu'après preuve de paiement ou confirmation cash autorisée.
- Les webhooks de paiement sont idempotents : un même événement fournisseur ne peut être traité qu'une fois.
- Une annulation applique la politique de la compagnie et peut déclencher un remboursement.
- Un ticket déjà embarqué ne peut pas être utilisé une seconde fois.

4. Modèle de domaine et données PostgreSQL

Le modèle relationnel est centré sur Company → Route → Trip → TripSeat → Booking. Les éléments sensibles au multi-tenancy portent explicitement company_id.

4.1 Vue d'ensemble

```
COMPANY
 ├─ AGENCY
 ├─ APP_USER
 ├─ VEHICLE — SEAT_LAYOUT — SEAT
 └─ ROUTE — TRIP — TRIP_SEAT — BOOKING — BOOKING_PASSENGER — TICKET
                              └─ PAYMENT — REFUND

AUDIT_EVENT et WEBHOOK_EVENT assurent traçabilité et idempotence.
```

4.2 Tables principales

Table	Champs structurants / rôle
company	Identité, licence, statut PENDING/APPROVED/SUSPENDED.
agency	Compagnie, ville, adresse, géolocalisation, téléphone.
app_user	Compte, company_id optionnel, rôle, statut.
city / station	Référentiel géographique.
route	Origine, destination, durée estimée, distance.
vehicle	Immatriculation, type, plan de sièges, statut.
seat_layout / seat	Configuration physique des sièges.
trip	Route, véhicule, agences, horaires, prix, statut.
trip_seat	Inventaire d'un trajet : état, hold, expiration.
booking	Référence, trajet, contact, statuts, montants.
booking_passenger	Passager, siège et tarif associé.
payment / refund	Fournisseur, méthode, montant, statuts, identifiants externes.
ticket	Numéro, QR, statuts check-in / boarding.

audit_event	Action, acteur, entité, métadonnées, horodatage.
webhook_event	provider_event_id unique, type, statut de traitement.

4.3 Contraintes recommandées

- UNIQUE sur les références de booking et ticket.
- UNIQUE sur provider_event_id pour les webhooks.
- Contraintes et verrouillage transactionnel sur trip_seat pour éviter la double vente.
- Index sur origin/destination/departure_at/status/company_id.
- Clés étrangères strictes et suppression contrôlée des entités métier.

5. Architecture backend et frontend

5.1 Choix d'architecture initial

> **Décision**
>
> Commencer par un modular monolith Spring Boot. Les frontières de modules sont propres dès le départ, mais on évite la complexité opérationnelle des microservices tant que le besoin n'est pas démontré.

5.2 Modules backend

```
mr.rihla
├─ identity
├─ company
├─ agency
├─ geography
├─ route
├─ vehicle
├─ trip
├─ inventory
├─ booking
├─ payment
├─ ticketing
├─ boarding
├─ notification
├─ reporting
├─ audit
└─ shared
```

Chaque module suit une structure api / application / domain / infrastructure afin de limiter le couplage et rendre une extraction future possible.

5.3 Stack backend

Technologie	Rôle
Java 21	Langage principal.
Spring Boot	API, configuration, sécurité et intégrations.

Spring Security	Authentification, RBAC et contrôle multi-tenant.
Spring Data JPA	Persistance relationnelle.
Spring Modulith	Frontières modulaires et événements internes.
PostgreSQL	Source de vérité transactionnelle.
Flyway	Versionnement du schéma.
Actuator	Santé et métriques techniques.
Testcontainers	Tests d'intégration sur PostgreSQL réel.
Docker	Packaging reproductible.

5.4 Frontend

- Angular 20+ avec composants standalone.
- Angular Material + Tailwind CSS pour l'interface.
- PWA mobile-first, français/arabe et RTL dès le départ.
- Zones /public, /operator et /admin dans un même frontend au MVP.
- Découpage ultérieur possible en Passenger PWA et Operator PWA si le produit grandit.

6. API REST et contrats

6.1 API publique

Méthode	Endpoint	Usage
GET	/api/public/cities	Référentiel des villes.
GET	/api/public/trips/search	Recherche par origine, destination, date, passagers.
GET	/api/public/trips/{tripId}	Détail d'un départ.
GET	/api/public/trips/{tripId}/seats	Disponibilité des sièges.
POST	/api/public/bookings	Création d'une réservation avec Idempotency-Key.
GET	/api/public/bookings/{reference}	Consultation sécurisée de réservation.
POST	/api/public/bookings/{reference}/cancel	Annulation selon politique.

6.2 API opérateur

Méthode	Endpoint
POST	/api/operator/routes

POST	/api/operator/trips
PATCH	/api/operator/trips/{id}
GET	/api/operator/trips/{id}/manifest
POST	/api/operator/bookings/counter
POST	/api/operator/tickets/{ticketId}/check-in
POST	/api/operator/tickets/{ticketId}/board
GET	/api/operator/dashboard

6.3 API admin

Méthode	Endpoint
GET	/api/admin/companies
POST	/api/admin/companies/{id}/approve
POST	/api/admin/companies/{id}/suspend
GET	/api/admin/audit

6.4 Principes de contrat

- DTO explicites, jamais d'entités JPA exposées directement.
- Erreurs structurées : code métier, message localisable, requestId.
- Pagination standardisée pour listes opérateur/admin.
- Idempotency-Key sur les écritures critiques.
- Versionnement API préparé si des clients mobiles tiers apparaissent.

7. Paiements, billets QR et notifications

7.1 Abstraction de paiement

```
PaymentProvider
 ├─ MockPaymentProvider    (MVP)
 └─ MoosylPaymentProvider  (Phase 2)
```

Le frontend ne décide jamais qu'un paiement est réussi. La confirmation est reçue ou vérifiée côté serveur, puis la réservation est confirmée de manière idempotente.

7.2 Cash et wallets

- CASH reste un moyen de paiement de première classe pour les ventes en agence.
- Les wallets locaux sont intégrés via un PaymentProvider dédié lorsque les contrats/API sont disponibles.
- Une réservation cash peut être AWAITING_PAYMENT jusqu'à confirmation par un agent autorisé.

7.3 Webhooks

```
Provider → POST /api/webhooks/payments/{provider}
  1. Vérifier signature / timestamp
  2. Vérifier providerEventId UNIQUE
  3. Mettre à jour Payment
  4. Confirmer Booking si applicable
  5. Répondre 2xx même sur événement déjà traité
```

7.4 Billet QR

Le QR transporte une référence minimale signée (ticketId, tripId, nonce/signature), pas les données personnelles du passager. Une signature asymétrique pourra être adoptée pour la validation offline future.

7.5 Notifications

- Réservation confirmée.
- Rappel 24 h et 2 h avant départ.
- Retard ou changement de point de départ.
- Annulation de trajet.
- Remboursement terminé.

Canaux possibles : in-app, email, SMS, WhatsApp et push. Au MVP, les fournisseurs externes peuvent être simulés derrière une interface NotificationProvider.

8. Kafka et architecture événementielle

8.1 Quand introduire Kafka

> **Règle**
>
> Kafka n’est pas nécessaire au MVP transactionnel. Il est introduit en Phase 2 lorsque notifications, audit et analytics justifient réellement un bus événementiel.

8.2 Événements

```
BookingCreated / BookingConfirmed / BookingExpired / BookingCancelled
PaymentSucceeded / PaymentFailed / PaymentRefunded
TripDelayed / TripCancelled / TripDeparted
PassengerCheckedIn / PassengerBoarded
```

8.3 Topics possibles

- booking.events
- payment.events
- trip.events
- boarding.events
- notification.commands

8.4 Transactional Outbox

```
Transaction PostgreSQL
  UPDATE booking SET status = CONFIRMED
  INSERT outbox_event(...)
COMMIT
        ↓
Outbox Publisher
        ↓
Kafka / Redpanda
        ↓
Notifications / Audit / Analytics
```

Cette stratégie évite le scénario dangereux “DB commit réussie mais publication Kafka échouée”.

9. Sécurité, multi-tenancy et conformité

9.1 Authentification

- Access token court pour les opérateurs.
- Refresh token HttpOnly avec rotation.
- Mot de passe hashé via Argon2 ou BCrypt correctement paramétré.
- MFA recommandé pour COMPANY_OWNER et PLATFORM_ADMIN.

9.2 Autorisation

Le RBAC ne suffit pas : toute ressource appartenant à une compagnie doit aussi être validée par company_id. Un COMPANY_MANAGER d’une compagnie ne peut jamais manipuler un trip d’une autre compagnie.

9.3 Protections prioritaires

Zone	Protection
Login / booking lookup	Rate limiting par IP, téléphone et compte.
Paiement	HMAC, timestamp, anti-replay, idempotence.
QR	Signature cryptographique, ticket/trip/status vérifiés.
API	Validation d’entrée, erreurs non bavardes, security headers.
Audit	Traçabilité des actions sensibles sans secrets ni données excessives.
Secrets	Variables sécurisées / Secret Manager, jamais dans Git.

9.4 Données personnelles

Collecter uniquement ce qui est nécessaire au service : identité minimale du passager, téléphone de contact et informations de voyage. Les journaux ne doivent pas contenir de données sensibles inutiles.

9.5 Conformité métier

Si le produit devient commercial, les obligations réglementaires concernant les opérateurs de transport, licences, intermédiation, paiement et responsabilité doivent être validées avec les autorités et/ou un conseil juridique local avant mise en service réelle.

10. Tests, concurrence et qualité

10.1 Pyramide de tests

Niveau	Outils	Cible
Unitaires	JUnit 5 / Mockito	Policies, services, règles métier.
Intégration	Testcontainers PostgreSQL	Flyway, JPA, locks, contraintes, Security.
Frontend	Angular tests	Composants, services, formulaires.
E2E	Playwright	Parcours passager et opérateur.
Charge	k6	Recherche, réservation concurrente, agences.

10.2 Tests incontournables

- 100 utilisateurs tentent le dernier siège : 1 succès, 99 SEAT_UNAVAILABLE.
- Webhook paiement reçu 5 fois : une seule confirmation.
- Deux agences tentent de vendre le même siège : une seule vente.
- Paiement réussi après expiration d'un hold : traitement cohérent, pas de double réservation.
- QR déjà utilisé : deuxième scan refusé.
- Trip annulé : réservations et notifications suivent la politique définie.

10.3 Critères de qualité

- Build reproductible.
- Migrations Flyway vérifiées en CI.
- Aucun secret dans le dépôt.
- Audit des dépendances runtime.
- Tests de concurrence sur PostgreSQL réel.
- CI verte obligatoire avant déploiement production.

11. Observabilité, performance et déploiement

11.1 Observabilité

- Spring Actuator pour health/readiness.
- Logs structurés avec requestId, bookingReference, tripId, companyId.
- Métriques : latence, taux de réservation, expirations de hold, échecs paiement, pool DB, lag Kafka.
- Jamais de secrets ou données personnelles complètes dans les logs.

11.2 Performance

PostgreSQL suffit au MVP pour recherche et transactions. Redis et OpenSearch ne sont pas introduits tant qu'un besoin mesuré ne le justifie pas.

11.3 Déploiement cible

```
Porkbun / DNS
   ├─ www.rihla.mr → Firebase Hosting → Angular PWA
   └─ api.rihla.mr → Cloud Run → Spring Boot → PostgreSQL

GitHub Actions → OIDC/WIF → Artifact Registry / Firebase Hosting
Secret Manager → secrets runtime
```

11.4 CI/CD

- Pull Request : tests, build, audits, aucun credential production.
- Push main : CI complète, pas de déploiement automatique obligatoire.
- Production : workflow_dispatch manuel et contrôlé.
- Images backend taguées par SHA Git pour traçabilité.

12. Roadmap Phase 0 → Production

Phase	Objectif
Phase 0 — Discovery	Interviewer 3–5 compagnies, 10–20 voyageurs et agents. Documenter ventes, annulations, tarifs, règles de sièges et paiements.
Phase 1 — Fondations	Spring Boot, PostgreSQL, Flyway, Security, Company, Agency, City, Vehicle, Route. Angular FR/AR + layout + auth opérateur.
Phase 2 — Trips & Inventory	Trip, TripSeat, SeatLayout, Fare, locking, hold et expiration.
Phase 3 — Booking	Booking, Passenger, Ticket, recherche → siège → réservation avec MockPayment.
Phase 4 — Operator	Dashboard, trajets, véhicules, routes, ventes guichet, manifeste.
Phase 5 — Boarding	QR, scanner, check-in, boarding, contrôle des doublons.
Phase 6 — Paiements réels	PaymentProvider, intégration fournisseur, webhooks, idempotence, refunds.
Phase 7 — Kafka	Redpanda/Kafka, Transactional Outbox, notifications, audit/analytics async.
Phase 8 — Production hardening	Rate limits, headers, monitoring, load tests, CI/CD, backups.
Phase 9 — Avancé	Offline boarding, push/WhatsApp, live trip status, GPS/ETA, fidélité si besoin réel.

12.1 Ce qui reste volontairement hors MVP

- Microservices et Kubernetes.
- Redis et Elasticsearch/OpenSearch.

- GPS live et ETA temps réel.
- IA et dynamic pricing.
- Loyalty, chat, reviews et wallet interne.

13. Critères de succès du projet

13.1 Succès produit

- Un voyageur peut trouver un départ pertinent en moins de quelques étapes.
- Une compagnie peut publier un départ et vendre au guichet sans double réservation.
- Le même inventaire est partagé entre ventes web et agence.
- Un billet QR peut être validé et embarqué de manière sûre.
- Le modèle FR/AR et mobile-first est utilisable sans imposer la création d'un compte.

13.2 Succès technique

- Toutes les règles critiques sont couvertes par des tests PostgreSQL réels.
- La concurrence sur le dernier siège est prouvée par test.
- Les paiements/webhooks sont idempotents.
- Le multi-tenancy est testé négativement entre compagnies.
- Le déploiement est reproductible par CI/CD avec secrets externes.
- Kafka n'est introduit qu'avec un cas d'usage asynchrone démontré.

13.3 Valeur portfolio

> **Positionnement portfolio**
>
> Rihla MR démontre un produit utile et complet : transactions concurrentes, multi-tenancy, paiements, webhooks, QR, PWA, architecture modulaire, Kafka/Outbox, tests d'intégration et CI/CD cloud — sans sacrifier l'utilité métier au profit de la technologie.

FIN DU DOSSIER DE CONCEPTION — RIHLA MR

Prochaine étape recommandée : Product Requirements (Epics → User Stories → Critères d'acceptation) puis contrat API détaillé.